
ΠΡΑΚΤΙΚΟΣ ΟΔΗΓΟΣ

Backend από την Αρχή

Μάθε backend φτιάχνοντας ένα πραγματικό API — βήμα προς βήμα, από το πρώτο endpoint μέχρι την παραγωγή.

Αυτός ο οδηγός σε πάει από «ξέρω λίγο προγραμματισμό» στο «έχω χτίσει και δημοσιεύσει ένα ολοκληρωμένο backend». Δεν διαβάζεις απλώς θεωρία: σε κάθε ενότητα προσθέτεις ένα κομμάτι σε ένα και μόνο έργο, ώστε στο τέλος να έχεις κάτι αληθινό στα χέρια σου.

Το έργο μας: ένα API καταγραφής εξόδων (expense tracker).

από τον **Ιωάννη Βόγγελη**

Διατίθεται δωρεάν στο πλαίσιο του προσωπικού μου brand.

Περιεχόμενα

Πώς να διαβάσεις αυτόν τον οδηγό

Τι χρειάζεσαι πριν ξεκινήσεις

ΜΕΡΟΣ Α · Θεμέλια

1. Τι κάνει πραγματικά ένα backend
2. Η γλώσσα του ιστού: HTTP, JSON, κωδικοί κατάστασης
3. Σχεδιάζοντας ένα καθαρό REST API
4. Το framework: γιατί και ποιο

ΜΕΡΟΣ Β · Δεδομένα & Ασφάλεια

5. Βάσεις δεδομένων και μοντελοποίηση
6. Ταυτοποίηση και εξουσιοδότηση
7. Επικύρωση εισόδου και χειρισμός σφαλμάτων

ΜΕΡΟΣ Γ · Κλίμακα & Παραγωγή

8. Αρχεία και εξωτερικές υπηρεσίες
9. Ασύγχρονη εργασία: ουρές και workers
10. Προσωρινή αποθήκευση (caching)
11. Δοκιμές (testing)
12. Δημοσίευση και παρατηρησιμότητα

Τι κάνεις μετά

Πώς να διαβάσεις αυτόν τον οδηγό

Ο πιο γρήγορος τρόπος να μην μάθεις backend είναι να παρακολουθείς το ένα tutorial μετά το άλλο χωρίς ποτέ να χτίζεις κάτι δικό σου. Γι' αυτό αυτός ο οδηγός είναι δομημένος αντίστροφα: κάθε κεφάλαιο τελειώνει με ένα μικρό βήμα σε ένα ενιαίο έργο. Στην αρχή θα είναι ένα ελάχιστο API· στο τέλος θα έχει βάση δεδομένων, χρήστες, ασφάλεια, αρχεία, ασύγχρονες εργασίες, cache, δοκιμές και θα τρέχει ζωντανά στο διαδίκτυο.

Διάλεξα ένα **API καταγραφής εξόδων** επειδή είναι αρκετά απλό ώστε να μην σε μπερδέψει, αλλά αρκετά πλούσιο ώστε να αγγίξει σχεδόν κάθε ιδέα που συναντά μια πραγματική υπηρεσία: λογαριασμούς χρηστών, σχέσεις μεταξύ δεδομένων, ανέβασμα αρχείων (αποδείξεις), κλήσεις σε εξωτερικές υπηρεσίες (email), αναφορές που τρέχουν στο παρασκήνιο και δεδομένα που αξίζει να αποθηκεύονται προσωρινά.

Δεν θα σου εξασφαλίσει δουλειά από μόνο του. Θα σου δώσει όμως τα θεμέλια που χρειάζεσαι για να χτίσεις κάτι μεγαλύτερο που θα το κάνει — και, το πιο σημαντικό, θα σου δείξει πώς τα κομμάτια κουμπώνουν μεταξύ τους.

Τι χρειάζεσαι πριν ξεκινήσεις

Ο οδηγός υποθέτει ότι μπορείς ήδη να γράψεις λίγο κώδικα — όχι σε προχωρημένο επίπεδο, απλώς τα βασικά. Αν δεν ξέρεις τι είναι μια συνάρτηση, ένας βρόχος ή μια συνθήκη, σταμάτησε εδώ και μάθε πρώτα μια γλώσσα. Θα σου πάρει λιγότερο χρόνο απ' όσο νομίζεις και όλα τα υπόλοιπα θα βγάζουν μετά νόημα.

Για γλώσσα, δύο ρεαλιστικές επιλογές:

- **Python** — ο πιο ομαλός δρόμος για να μπεις. Καθαρή σύνταξη, τεράστιο οικοσύστημα, και το framework που θα χρησιμοποιήσουμε εδώ (FastAPI) είναι Python. Την προτιμούν πολλές startups και ομάδες δεδομένων/τεχνητής νοημοσύνης.
- **Java** — πιο απαιτητική στην αρχή, αλλά πανταχού παρούσα σε μεγάλους οργανισμούς. Χτίζει γερές αρχές αντικειμενοστρέφειας και το Spring Boot παραμένει μία από τις πιο περιζήτητες δεξιότητες backend.

Δεν έχει σημασία ποια θα διαλέξεις: οι έννοιες που θα μάθεις μεταφέρονται σχεδόν αυτούσιες από τη μία στην άλλη. Τα παραδείγματα εδώ είναι σε Python/FastAPI, αλλά η λογική είναι κοινή.

ΜΕΡΟΣ Α — ΘΕΜΕΛΙΑ

1. Τι κάνει πραγματικά ένα backend

Κάθε εφαρμογή που έχεις χρησιμοποιήσει έχει δύο πλευρές. Η μία είναι το **frontend**: ό,τι βλέπεις και ακουμπάς — κουμπιά, οθόνες, χρώματα. Η άλλη είναι το **backend**: ό,τι δεν βλέπεις — πού φυλάσσονται τα δεδομένα, η λογική που αποφασίζει τι επιτρέπεται, και οι κανόνες που κρατούν τα πάντα σωστά και ασφαλή.

Πάρε το δικό μας παράδειγμα. Πατάς «Αποθήκευση» σε ένα έξοδο των 12€ για καφέ. Το frontend απλώς σχεδιάζει μια νέα γραμμή στη λίστα σου. Το backend είναι αυτό που στην πραγματικότητα ελέγχει ότι το ποσό είναι έγκυρο, το συνδέει με τον δικό σου λογαριασμό και όχι κάποιου άλλου, το γράφει μόνιμα ώστε να υπάρχει και αύριο, και φροντίζει να μην καταχωρηθεί δύο φορές αν πατήσεις κατά λάθος διπλά. Ως backend μηχανικός, χτίζεις ακριβώς αυτή τη δεύτερη πλευρά.

Client και server

Στον πυρήνα του, ο ιστός είναι μια συνομιλία δύο υπολογιστών. Ο ένας ρωτάει, ο άλλος απαντάει. Ο υπολογιστής που ρωτάει λέγεται **client** (πελάτης): το κινητό σου, ο browser, μια εφαρμογή. Ο υπολογιστής που απαντάει λέγεται **server** (διακομιστής) — ένας υπολογιστής που είναι μονίμως ανοιχτός και περιμένει αιτήματα για να τα εξυπηρετήσει.

Δεν υπάρχει τίποτα μαγικό σε έναν server· είναι απλώς ένας υπολογιστής με μια συγκεκριμένη δουλειά: να κάθεται και να απαντάει. Ο φορητός σου θα μπορούσε να είναι server. Ένα τεράστιο μηχάνημα σε data center κάποιου παρόχου cloud είναι επίσης server. Το 99% των εφαρμογών που έχεις αγγίξει είναι, στην ουσία, ένας client που ζητάει κάτι κι ένας server που απαντά. Όλη σου η δουλειά είναι να φτιάξεις αυτό που απαντά.

Το API είναι ο κατάλογος των πράξεων

Σκέψου τον πάγκο ενός ταχυδρομείου. Δεν μπαίνεις στο πίσω δωμάτιο να ψάξεις μόνος σου· υπάρχει ένας συγκεκριμένος κατάλογος με το τι μπορείς να ζητήσεις (στείλε δέμα, αγόρασε γραμματόσημο, πλήρωσε λογαριασμό) και ένας ορισμένος τρόπος να το ζητήσεις. Ένα **API** (Application Programming Interface) είναι ακριβώς αυτός ο πάγκος για τον server σου: η λίστα με τις πράξεις που δέχεται και οι κανόνες για το πώς ζητιέται η καθεμία.

Όταν λοιπόν λέμε «θα φτιάξεις ένα API», εννοούμε ότι θα ορίσεις αυτόν τον κατάλογο — τι ξέρει να κάνει η υπηρεσία σου και πώς κάποιος της το ζητάει σωστά. Το κινητό σου δεν μιλάει ποτέ κατευθείαν στη βάση δεδομένων (θα ήταν επικίνδυνο). Ζητάει από το API «δώσε μου τα έξοδα αυτού του χρήστη» και το API απαντά σε μια καθαρή, προβλέψιμη μορφή.

Η ζωή ενός αιτήματος

Ας ακολουθήσουμε ένα αίτημα από την αρχή ως το τέλος:

- Ο client φτιάχνει ένα αίτημα: μια διεύθυνση (URL) που λέει πού πάει, μια μέθοδο που λέει τι θέλει, και ίσως λίγα δεδομένα.
- Το αίτημα ταξιδεύει μέσα από το δίκτυο ως τη διεύθυνση του server.
- Ο server ακούει σε μια συγκεκριμένη **θύρα (port)** και λαμβάνει το αίτημα.
- Ο server τρέχει τον κώδικά σου για να αποφασίσει τι πρέπει να γίνει: να διαβάσει κάτι από τη βάση, να ελέγξει ένα δικαίωμα, οτιδήποτε.
- Ο server συνθέτει μια απάντηση και τη στέλνει πίσω μέσα από το δίκτυο.
- Ο client παίρνει την απάντηση και κάνει κάτι με αυτήν — π.χ. τη δείχνει στην οθόνη.



Δύο λέξεις που θα ακούς συνεχώς όσο δουλεύεις τοπικά: **localhost**, το ψευδώνυμο που σημαίνει «αυτός εδώ ο υπολογιστής» — όταν τρέχεις τον server στον φορητό σου, είσαι ταυτόχρονα και ο client και ο server· και **θύρα (port)**, ένας αριθμημένος «πάγκος» στο μηχάνημα. Το `localhost:8000` σημαίνει απλώς «ο server που τρέχει εδώ, πίσω από την πόρτα 8000».

2. Η γλώσσα του ιστού: HTTP, JSON, κωδικοί κατάστασης

Το **HTTP** είναι το σύνολο των κανόνων με τους οποίους μιλάνε clients και servers. Κάθε αίτημα έχει μερικά μέρη· αξίζει να καταλάβεις το καθένα, γιατί όλη η υπόλοιπη δουλειά χτίζεται πάνω τους.

Μέθοδοι

Η μέθοδος δηλώνει την *πρόθεσή σου*. Οι λίγες που θα χρησιμοποιείς αντιστοιχούν καθαρά στα τέσσερα πράγματα που κάνεις ποτέ με δεδομένα — δημιουργία, ανάγνωση, ενημέρωση, διαγραφή:

Μέθοδος	Τι κάνει	Παράδειγμα στο έργο μας
GET	Διαβάζει δεδομένα (δεν αλλάζει τίποτα)	Φέρει όλα τα έξοδα του χρήστη
POST	Δημιουργεί κάτι νέο	Καταχώρησε ένα νέο έξοδο
PUT	Αντικαθιστά πλήρως ένα στοιχείο	Αντικατέστησε ολόκληρο ένα έξοδο
PATCH	Αλλάζει ένα μέρος ενός στοιχείου	Διόρθωσε μόνο το ποσό ενός εξόδου
DELETE	Αφαιρεί κάτι	Σβήσε ένα έξοδο

Ο σημαντικός κανόνας: κράτα αυτές τις προθέσεις ειλικρινείς. Ένα GET δεν πρέπει ποτέ να αλλάζει κάτι· ένα DELETE αφαιρεί. Αν τις σεβαστείς, το API σου γίνεται προβλέψιμο — και αυτό είναι η αρχή του καλού σχεδιασμού.

Διευθύνσεις (URLs) και διαδρομές

Το URL είναι η διεύθυνση αυτού που θέλεις. Ας το σπάσουμε: στο `https://myapp.com/expenses/42` το `https://` είναι το πρωτόκολλο (HTTP με κρυπτογράφηση), το `myapp.com` είναι η διεύθυνση του server, και το `/expenses/42` είναι η **διαδρομή (path)** που δείχνει σε έναν συγκεκριμένο πόρο — εδώ, το έξοδο με αναγνωριστικό 42. Η διαδρομή αποτυπώνει τη δομή του API σου: `/expenses` σημαίνει «όλα τα έξοδα», `/expenses/42` σημαίνει «αυτό το ένα». Εσύ τα σχεδιάζεις.

Κωδικοί κατάστασης

Κάθε απάντηση συνοδεύεται από έναν τριψήφιο κωδικό που λέει πώς πήγε. Δεν τους αποστηθίζεις όλους· μαθαίνεις τις οικογένειες και μερικούς συνηθισμένους:

Οικογένει α	Σημασία	Συχνά παραδείγματα
2xx	Επιτυχία	200 OK · 201 Created
3xx	Ψάξε αλλού / ανακατεύθυνση	301 · 304 Not Modified
4xx	Λάθος του client	400 Bad Request · 401 · 403 · 404

Οικογένει α	Σημασία	Συχνά παραδείγματα
5xx	Λάθος του server	500 Internal Error · 503

Με δυο λόγια: 4xx σημαίνει «εσύ, ο client, έκανες κάτι λάθος» — ένα 404 λέει ότι ζήτησες κάτι που δεν υπάρχει. 5xx σημαίνει «ο server τα θαλάσσωσε» ενώ προσπαθούσε να σε εξυπηρετήσει.

Κεφαλίδες (headers) και το σώμα

Οι **κεφαλίδες** είναι μεταδεδομένα που ταξιδεύουν μαζί με κάθε αίτημα και απάντηση, χωριστά από τα κύρια δεδομένα. Δεν τις αποστηθίζεις· αρκεί να ξέρεις ότι υπάρχουν. Δύο που θα δεις συνέχεια: Content-Type: application/json (λέει σε τι μορφή είναι τα δεδομένα) και Authorization: Bearer <token> (αποδεικνύει ποιος είσαι — θα το φτιάξουμε στην ενότητα της ταυτοποίησης).

Το **σώμα (body)** είναι τα ίδια τα δεδομένα. Όταν δημιουργείς ένα έξοδο, οι λεπτομέρειές του ταξιδεύουν στο σώμα του αιτήματος. Σχεδόν πάντα έχει μορφή **JSON** — απλά ζευγάρια κλειδιού/τιμής, αναγνώσιμα και από ανθρώπους και από μηχανές:

```
{
  "περιγραφή": "Καφές",
  "ποσό": 3.50,
  "κατηγορία": "φαγητό",
  "ημερομηνία": "2026-05-14"
}
```

Τα άγκιστρα κρατούν ένα αντικείμενο· κάθε κλειδί δείχνει σε μια τιμή. Αυτό είναι όλο. Σχεδόν κάθε σύγχρονο API μιλάει JSON, οπότε θα το διαβάζεις και θα το γράφεις συνέχεια.

3. Σχεδιάζοντας ένα καθαρό REST API

Το **REST** δεν είναι τεχνολογία· είναι ένα δημοφιλές, λογικό στυλ για να οργανώσεις τα endpoints σου. Δεν χρειάζεσαι θεωρία — μόνο δύο κανόνες:

- **Οργάνωσε το API γύρω από πόρους (ουσιαστικά), όχι ενέργειες (ρήματα).** Ο κόσμος του έργου μας έχει έξοδα, άρα ο πόρος είναι `/expenses`. Δεν φτιάχνεις διαδρομές όπως `/getExpenses` ή `/createExpense`.
- **Άσε τη μέθοδο HTTP να πει την ενέργεια.** Το URL ονομάζει το «τι» (τον πόρο), η μέθοδος λέει το «πώς» (διάβασε, δημιούργησε, σβήσε).

Έτσι, ένα καλοσχεδιασμένο API για τα έξοδα μοιάζει κάπως ως εξής:

Μέθοδος + Διαδρομή	Τι κάνει
GET <code>/expenses</code>	Επιστρέφει όλα τα έξοδα
POST <code>/expenses</code>	Δημιουργεί ένα νέο έξοδο
GET <code>/expenses/42</code>	Επιστρέφει το έξοδο 42
PATCH <code>/expenses/42</code>	Ενημερώνει μέρος του εξόδου 42
DELETE <code>/expenses/42</code>	Διαγράφει το έξοδο 42

Πρόσεξε ότι η ίδια διαδρομή, `/expenses/42`, κάνει εντελώς διαφορετικά πράγματα ανάλογα με τη μέθοδο. Αυτό είναι το REST: καθαρό, προβλέψιμο, και το αναγνωρίζει αμέσως κάθε backend μηχανικός. Είναι όση σχεδίαση API χρειάζεσαι για να ξεκινήσεις.

4. Το framework: γιατί και ποιο

Σχεδόν καμία ομάδα δεν γράφει από το μηδέν τη διαχείριση δικτύου, την ανάλυση JSON και τη δρομολόγηση αιτημάτων. Χρησιμοποιεί ένα **framework**: μια εργαλειοθήκη που αναλαμβάνει όλα τα επαναλαμβανόμενα κομμάτια, ώστε εσύ να επικεντρωθείς στη δική σου λογική.

Framework	Γλώσσα	Πού το βλέπεις
FastAPI	Python	Startups, ομάδες AI/δεδομένων
Spring Boot	Java	Μεγάλοι οργανισμοί, enterprise
Express	Node.js	Startups, full-stack JS

Μην προσπαθήσεις να μάθεις και τα τρία. Διάλεξε ένα και εμβάθυνε. Εδώ χρησιμοποιούμε **FastAPI** γιατί είναι γρήγορο να το πιάσεις και σου δίνει αυτόματα μια διαδραστική σελίδα δοκιμών.

Το πιο σημαντικό μάθημα ολόκληρης αυτής της ενότητας: μην παρακολουθήσεις δώδεκα tutorials χωρίς να στείλεις ούτε ένα αίτημα. Διάλεξε ένα framework, φτιάξε κάτι αληθινό, και οι έννοιες που θα μάθεις θα μεταφερθούν σε οποιοδήποτε άλλο μέσα σε λίγες μέρες.

Αυτές οι μεταφάσεις έννοιες είναι:

- **Δρομολόγηση (routing):** αντιστοίχιση μιας μεθόδου + διαδρομής (π.χ. GET /expenses) σε μια συγκεκριμένη συνάρτηση του κώδικά σου.
- **Middleware:** κώδικας που τρέχει πριν ή μετά από κάθε αίτημα (καταγραφή, έλεγχος ταυτότητας, κ.λπ.).
- **Επικύρωση (validation):** εξασφάλιση ότι ένα εισερχόμενο αίτημα έχει τη σωστή μορφή προτού το αγγίξει η λογική σου.
- **Χειρισμός σφαλμάτων:** εντοπισμός αποτυχιών και επιστροφή καθαρής απάντησης αντί για ωμό σφάλμα.
- **Μεταβλητές περιβάλλοντος:** κράτημα ρυθμίσεων και μυστικών (κωδικοί, κλειδιά) έξω από τον κώδικα.

Δεν χρειάζεται να τα εμβαθύνεις τώρα. Κατάλαβέ τα σε γενικές γραμμές, χτίσε ένα αληθινό backend, και το βάθος θα έρθει μόνο του με την καθημερινή χρήση.

♦ Έργο — Βήμα 1: το πρώτο σου API

Φτιάξε ένα απλό API εξόδων με FastAPI. Δεν χρειάζεσαι ακόμη βάση δεδομένων — κράτα τα έξοδα σε μια λίστα Python στη μνήμη. Πρόσθεσε endpoints για: δημιουργία εξόδου (POST /expenses), λίστα όλων (GET /expenses) και ενημέρωση ενός (PATCH /expenses/{id}).

Ο στόχος εδώ δεν είναι ο «έξυπνος» κώδικας· είναι η καλωδίωση. Τρέξε τον server, άνοιξε το localhost:8000/docs και πάτα τα κουμπιά για να στείλεις αληθινά αιτήματα. Μόλις κάνεις ring στο δικό σου API και σου γυρίσει JSON, έχεις κάνει πολύ περισσότερα απ' όσο νομίζεις.

Οδηγός: επίσημη τεκμηρίωση FastAPI — fastapi.tiangolo.com

ΜΕΡΟΣ Β — ΔΕΔΟΜΕΝΑ & ΑΣΦΑΛΕΙΑ

5. Βάσεις δεδομένων και μοντελοποίηση

Αυτή τη στιγμή το API σου ξεχνά τα πάντα μόλις κλείσει, γιατί κρατάει τα έξοδα σε μια λίστα στη μνήμη. Μια **βάση δεδομένων** είναι ένα πρόγραμμα του οποίου όλη η δουλειά είναι να αποθηκεύει δεδομένα μόνιμα και να σου επιτρέπει να τα ανακτάς γρήγορα. Το να τη μάθεις καλά είναι από τα πράγματα με τη μεγαλύτερη αξία που μπορείς να κάνεις ως backend μηχανικός.

Ξεκίνα με **SQL**. Πάντα. Οι σχεσιακές βάσεις SQL αποθηκεύουν δεδομένα σε πίνακες — σαν πολύ ισχυρά λογιστικά φύλλα — και τις ρωτάς με μια γλώσσα που λέγεται επίσης SQL. Οι βάσεις NoSQL έχουν κι αυτές τη θέση τους, αλλά μια σχεσιακή βάση σε αναγκάζει να σκεφτείς σωστά τη δομή των δεδομένων σου· μόλις το μάθεις αυτό, το NoSQL είναι εύκολο. Το αντίστροφο δεν ισχύει. Πρακτικά, χρησιμοποίησε **PostgreSQL** (όλοι το λένε «Postgres»): δωρεάν, παντού, και τρέχει τεράστιο μέρος της βιομηχανίας.

Οι έννοιες που πρέπει να καταλάβεις

Έννοια	Τι είναι
Πίνακες & γραμμές	Ένας πίνακας είναι σαν λογιστικό φύλλο· κάθε γραμμή είναι μία εγγραφή (π.χ. ένα έξοδο).
Στήλες	Τα πεδία κάθε γραμμής (ποσό, περιγραφή, ημερομηνία).
Πρωτεύον κλειδί	Το μοναδικό αναγνωριστικό κάθε γραμμής, ώστε να δείχνεις ακριβώς σε μία.
Ξένο κλειδί	Μια στήλη που αναφέρεται σε γραμμή άλλου πίνακα (π.χ. ένα έξοδο ανήκει σε έναν χρήστη).
Ευρετήρια (indexes)	Δομές που κάνουν τις αναγνώσεις γρήγορες (και τις εγγραφές λίγο πιο αργές).
Ενώσεις (joins)	Συνδυασμός σχετικών δεδομένων από δύο ή περισσότερους πίνακες σε ένα ερώτημα.
Περιορισμοί (constraints)	Κανόνες που επιβάλλει η βάση (π.χ. «όχι κενό», «μοναδικό»).
Migrations	Ελεγχόμενες αλλαγές στη δομή των πινάκων με την πάροδο του χρόνου.
Συναλλαγές (transactions)	Ομάδα ενεργειών που ή πετυχαίνουν όλες μαζί ή αποτυγχάνουν όλες μαζί.

Τα ευρετήρια και οι ενώσεις είναι εκεί που μπερδεύονται οι περισσότεροι αρχάριοι, και οι συναλλαγές είναι εκεί που κρύβονται τα χειρότερα bugs. Φαντάσου να μεταφέρεις χρήματα μεταξύ δύο λογαριασμών και το δεύτερο βήμα να αποτύχει: μια συναλλαγή εγγυάται ότι δεν θα μείνεις ποτέ στη μέση με χαμένα χρήματα.

Το πιο σημαντικό: το να συνδέσεις και να τρέξεις μια βάση είναι το εύκολο μέρος. Το δύσκολο — και το πιο κρίσιμο — είναι η **μοντελοποίηση**: να αποφασίσεις ποιους πίνακες χρειάζεσαι, ποιες στήλες έχουν και πώς σχετίζονται. Οποιοσδήποτε τρέχει ένα CREATE TABLE· το να το σχεδιάσεις σωστά είναι η πραγματική δεξιότητα. Αφιέρωσε περισσότερο χρόνο στο σχήμα παρά στον κώδικα σύνδεσης.

♦ Έργο — Βήμα 2: πραγματική βάση δεδομένων

Βάλε μια αληθινή Postgres πίσω από το API σου. Πριν γράψεις κώδικα, σχεδίασε τους πίνακες **σε χαρτί**. Χρειάζεσαι τουλάχιστον: χρήστες (users), έξοδα (expenses) και κατηγορίες (categories). Αποφάσισε πρωτεύοντα κλειδιά, ξένα κλειδιά και σχέσεις: ένας χρήστης έχει πολλά έξοδα· μια κατηγορία ομαδοποιεί πολλά έξοδα.

Μετά σύνδεσέ την με το FastAPI και μετέφερε τα δεδομένα από τη λίστα στη μνήμη μέσα στη βάση. Αφιέρωσε τον χρόνο σου στο σχήμα, όχι στον κώδικα σύνδεσης.

Τα δεδομένα σου είναι πλέον μόνιμα — αλλά προς το παρόν οποιοσδήποτε βρει το API μπορεί να διαβάσει και να γράψει τα έξοδα του καθενός. Ωρα να το κλειδώσουμε.

6. Ταυτοποίηση και εξουσιοδότηση

Δύο λέξεις που ακούγονται ίδιες αλλά σημαίνουν διαφορετικά πράγματα:

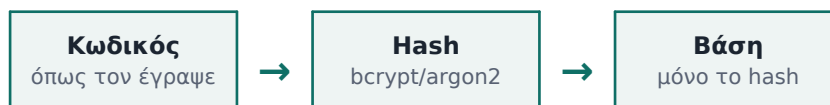
- **Ταυτοποίηση (authentication):** ποιος είσαι; (σύνδεση, απόδειξη ταυτότητας).
- **Εξουσιοδότηση (authorization):** τι σου επιτρέπεται να κάνεις; (δικαιώματα, έλεγχος πρόσβασης).

Πρώτα έρχεται η ταυτοποίηση (αποδεικνύεις ποιος είσαι) και μετά η εξουσιοδότηση αποφασίζει τι μπορείς να αγγίξεις. Αν το χειριστείς σωστά, είσαι ήδη μπροστά από πάρα πολλούς που το προσπερνούν και φτιάχνουν ανασφαλείς εφαρμογές.

Αποθήκευση κωδικών

Ποτέ, μα ποτέ, μην αποθηκεύεις κωδικούς ως απλό κείμενο. Αν διαρρεύσει η βάση σου, παραβιάζονται ακαριαία όλοι οι χρήστες. Αντ' αυτού αποθηκεύεις ένα **hash**: μια μονόδρομη, κρυπτογραφημένη εκδοχή του κωδικού, φτιαγμένη από αλγόριθμο όπως το bcrypt ή το argon2. «Μονόδρομη» σημαίνει ότι δεν μπορείς να ξαναγυρίσεις το hash σε κωδικό.

Στην εγγραφή, κάνεις hash τον κωδικό και αποθηκεύεις μόνο το hash. Στη σύνδεση, κάνεις hash αυτό που πληκτρολόγησε ο χρήστης και το συγκρίνεις με το αποθηκευμένο. Η βάση δεν γνωρίζει ποτέ τον πραγματικό κωδικό.



Sessions ή JWT;

Μόλις κάποιος συνδεθεί, χρειάζεσαι τρόπο να «θυμάσαι» ότι είναι συνδεδεμένος σε κάθε επόμενο αίτημα, αφού το ίδιο το HTTP ξεχνάει μεταξύ αιτημάτων. Δύο συνηθισμένες προσεγγίσεις:

	Sessions	JWT
Πού «ζει» η κατάσταση	Στον server (ή σε αποθήκη τύπου Redis)	Μέσα σε ένα token που κρατά ο client
Κλιμάκωση	Θέλει κοινή αποθήκη σε όλους τους servers	Χωρίς κατάσταση, εύκολη κλιμάκωση
Αποσύνδεση	Εύκολη — σβήνεις τη session	Πιο δύσκολη — το token ισχύει ως τη λήξη

Ένα **JWT** (JSON Web Token) είναι μια υπογεγραμμένη συμβολοσειρά που σου δίνει ο server στη σύνδεση. Την στέλνεις πίσω σε κάθε αίτημα για να αποδείξεις ποιος είσαι, και ο server επαληθεύει την υπογραφή χωρίς να ψάξει πουθενά — γι' αυτό κλιμακώνει τόσο καλά.

Cookies και OAuth

Ένα **cookie** είναι ένα μικρό κομμάτι δεδομένων που ο browser αποθηκεύει και στέλνει αυτόματα, συχνά για να κρατά ένα αναγνωριστικό session ή ένα token. Τρεις ρυθμίσεις έχουν σημασία για την ασφάλεια: `HttpOnly` (η JavaScript δεν μπορεί να το διαβάσει), `Secure` (στέλνεται μόνο μέσω HTTPS) και `SameSite` (περιορίζει την αποστολή μεταξύ ιστοτόπων).

Το **OAuth** είναι αυτό που τρέχει πίσω από το «Σύνδεση με Google» ή «με GitHub». Αντί να κρατάς εσύ τον κωδικό του χρήστη, αναθέτεις τη σύνδεση σε έναν πάροχο που εμπιστεύεσαι: στέλνεις τον χρήστη στην Google, η Google επιβεβαιώνει ποιος είναι και σου γυρίζει έναν κωδικό, κι εσύ τον ανταλλάσσεις με ένα token. Δεν βλέπεις ποτέ τον κωδικό του — ασφαλέστερο για όλους.

Έλεγχος πρόσβασης

Αφού μάθεις ποιος είναι κάποιος, αποφασίζεις τι του επιτρέπεται. Ένας χρήστης πρέπει να βλέπει και να επεξεργάζεται τα δικά του έξοδα και ποτέ κάποιου άλλου. Ακούγεται προφανές, αλλά το να το ξεχνάς είναι από τα πιο συχνά και επικίνδυνα σφάλματα του πραγματικού κόσμου: APIs που δίνουν χαρούμενα στον χρήστη Α όλα τα δεδομένα του χρήστη Β επειδή κανείς δεν έβαλε τον έλεγχο.

♦ Έργο — Βήμα 3: κλείδωσε το API

Πρόσθεσε εγγραφή και σύνδεση χρήστη. Κάνε hash τους κωδικούς, δώσε ένα token στη σύνδεση, και απαίτησέ το σε κάθε endpoint που αγγίζει δεδομένα.

Κυρίως, επίβαλε τον έλεγχο πρόσβασης: κάθε χρήστης βλέπει και αλλάζει μόνο τα δικά του έξοδα. Το API σου είναι πλέον κοντά σε ένα MVP.

Οδηγός: ενότητα ασφαλείας στην επίσημη τεκμηρίωση FastAPI.

7. Επικύρωση εισόδου και χειρισμός σφαλμάτων

Το μεγαλύτερο μέρος του έργου είναι έτοιμο· τώρα μπαίνουμε στις λεπτομέρειες που ξεχωρίζουν ένα demo από κάτι που αντέχει.

Επικύρωση εισόδου

Μην εμπιστεύεσαι ποτέ την είσοδο. Ποτέ. Οτιδήποτε έρχεται από client μπορεί να είναι λάθος, σκουπίδι ή ευθεία επίθεση. Τα περισσότερα κενά ασφαλείας σε αληθινά συστήματα οφείλονται σε κάποια μορφή **injection**, όπου ο εισβολέας στέλνει δεδομένα που ο κώδικάς σου εκλαμβάνει κατά λάθος ως εντολή αντί για απλά δεδομένα.

Το κλασικό παράδειγμα είναι το **SQL injection**: αν κολλήσεις είσοδο χρήστη κατευθείαν μέσα σε ένα ερώτημα SQL, κάποιος μπορεί να γράψει είσοδο που μετατρέπεται σε δικό του ερώτημα και να διαβάσει ή να σβήσει όλη σου τη βάση. Οι διορθώσεις δεν είναι περίπλοκες:

- Επικύρωσε κάθε εισερχόμενο αίτημα σε σχέση με ένα αναμενόμενο σχήμα. Στο FastAPI το Pydantic σου επιτρέπει να δηλώσεις ακριβώς πώς μοιάζει ένα έγκυρο αίτημα και να απορρίπτει αυτόματα τα υπόλοιπα.
- Χρησιμοποίησε παραμετροποιημένα ερωτήματα ή ένα ORM — ποτέ μην ενώνεις χειροκίνητα είσοδο χρήστη μέσα σε συμβολοσειρά SQL. Έτσι η βάση αντιμετωπίζει την είσοδο πάντα ως δεδομένα, ποτέ ως εντολή.
- Απόρριψε ό,τι δεν ταιριάζει σε αυτό που περιμένεις, αντί να προσπαθείς να το «καθαρίσεις».

Χειρισμός σφαλμάτων

Τα πράγματα θα πάνε στραβά· το ερώτημα είναι τι γίνεται τότε. Πρέπει να ισορροπήσεις δύο στόχους που τραβάνε αντίθετα:

- **Να είσαι χρήσιμος στον εαυτό σου.** Θέλεις αρκετές λεπτομέρειες στα logs σου ώστε να εντοπίζεις και να διορθώνεις ένα πρόβλημα γρήγορα.
- **Να μην αποκαλύπτεις τίποτα σε εισβολείς.** Το μήνυμα που γυρνάς στον client πρέπει να είναι καθαρό και γενικό. Ένα ωμό stack trace ή σφάλμα βάσης που φτάνει στον χρήστη είναι δώρο σε έναν κακόβουλο — του λέει ακριβώς πώς είναι φτιαγμένο το σύστημά σου.

Ο κανόνας είναι απλός: **κατέγραψε τα πάντα στον server, γύρνα ένα καθαρό μήνυμα στον client.** Ο χρήστης βλέπει «500 Internal Server Error»· τα logs σου κρατούν όλο το stack trace. Ποτέ το αντίστροφο.

♦ Έργο — Βήμα 4: κάνε το ανθεκτικό

Πρόσθεσε αυστηρή επικύρωση σε κάθε endpoint, ώστε τα κακοδιατυπωμένα αιτήματα να απορρίπτονται με ένα σαφές 400 και ένα χρήσιμο μήνυμα. Βεβαιώσου ότι χρησιμοποιείς παραμετροποιημένα ερωτήματα ή ORM παντού.

Πρόσθεσε έναν καθολικό χειριστή σφαλμάτων που καταγράφει όλες τις λεπτομέρειες στην πλευρά του server αλλά επιστρέφει καθαρά, γενικά μηνύματα στον καλούντα. Το backend σου είναι πλέον πιο ασφαλές για να εκτεθεί στον κόσμο.

ΜΕΡΟΣ Γ — ΚΛΙΜΑΚΑ & ΠΑΡΑΓΩΓΗ

8. Αρχεία και εξωτερικές υπηρεσίες

Τα πραγματικά backend σπάνια δουλεύουν μόνα τους. Αποθηκεύουν αρχεία και καλούν υπηρεσίες άλλων εταιρειών. Αυτή η ενότητα είναι για το να κάνεις και τα δύο με ασφάλεια.

Αποθήκευση αρχείων

Όταν οι χρήστες ανεβάζουν πράγματα — στην περίπτωσή μας, φωτογραφίες αποδείξεων — μην αποθηκεύεις τα ίδια τα αρχεία μέσα στη βάση ή στον δίσκο του server. Και τα δύο γεμίζουν, γίνονται αργά και δεν κλιμακώνουν. Αντ' αυτού χρησιμοποίησε ένα **object store**, μια υπηρεσία φτιαγμένη ειδικά για να κρατά αρχεία φθηνά και να τα σερβίρει γρήγορα. Το πρότυπο είναι το Amazon S3 (ή το MinIO για δωρεάν, τοπική χρήση συμβατή με S3).

Το μοτίβο: το αρχείο πάει στο object store κι εσύ αποθηκεύεις στη βάση σου *μόνο* το κλειδί του (τη διεύθυνσή του). Έτσι η βάση μένει μικρή και γρήγορη, και το βαρύ υλικό ζει αλλού. Φτιάξε ένα μοναδικό κλειδί για κάθε αρχείο (π.χ. με ένα UUID) ώστε δύο χρήστες που ανεβάζουν το receipt.jpg να μη συγκρουστούν.

Ασφαλείς κλήσεις σε APIs τρίτων

Τη στιγμή που καλείς την υπηρεσία μιας άλλης εταιρείας (έναν επεξεργαστή πληρωμών, έναν αποστολέα email, ένα API καιρού), εξαρτάσαι από κάτι που δεν ελέγχεις. Κάποιες φορές θα είναι αργό, κάποιες θα αποτυγχάνει. Σχεδίασε γι' αυτό:

- **Βάζε πάντα χρονικό όριο (timeout)**, ώστε να μην περιμένεις για πάντα μια υπηρεσία που κόλλησε.
- **Πρόσθεσε επαναλήψεις (retries)** για προσωρινές αποτυχίες, αλλά με αυξανόμενη αναμονή, για να μην πνίγεις μια υπηρεσία που ήδη ζορίζεται.
- **Κράτα τα κλειδιά API σε μεταβλητές περιβάλλοντος**, ποτέ γραμμένα μέσα στον κώδικα.
- **Απότυχε ομαλά** όταν η άλλη υπηρεσία είναι εκτός, αντί να αφήσεις να καταρρεύσει όλη η εφαρμογή σου.

♦ Έργο — Βήμα 5: αρχεία και email

Επίτρεψε στους χρήστες να ανεβάζουν μια φωτογραφία απόδειξης για κάθε έξοδο. Αποθήκευσε την εικόνα σε S3 (ή τοπικά με MinIO) και κράτα στη Postgres μόνο το κλειδί. Μετά ενσωμάτωσε ένα εξωτερικό API: στείλε ένα email καλωσορίσματος μέσω μιας υπηρεσίας όπως το Resend ή το SendGrid όταν εγγράφεται ένας χρήστης — με timeout και δυνατότητα επανάληψης.

Τώρα η εφαρμογή σου μιλάει με τον έξω κόσμο. Αλλά μέρος αυτής της δουλειάς είναι αργό, και το να περιμένει ο χρήστης είναι κακή εμπειρία. Ώρα για ουρές.

9. Ασύγχρονη εργασία: ουρές και workers

Κάποιες δουλειές δεν πρέπει να γίνονται όσο περιμένει ο χρήστης: αποστολή email, δημιουργία μιας μεγάλης μηνιαίας αναφοράς εξόδων, επεξεργασία μιας εικόνας. Αν τις κάνεις μέσα στο ίδιο το αίτημα, ο χρήστης κοιτάει ένα σπινάκι που γυρίζει και ο server σου μπλοκάρει σε αργή δουλειά αντί να εξυπηρετεί άλλους.

Η λύση είναι να μεταφέρεις την αργή δουλειά στο παρασκήνιο. Βάζεις μια εργασία σε μια **ουρά (queue)**, γυρνάς αμέσως απάντηση στον χρήστη, και μια ξεχωριστή διεργασία (**worker**) τραβάει εργασίες από την ουρά και τις εκτελεί όποτε είναι έτοιμη.



Αυτό σου δίνει δύο μεγάλα πλεονεκτήματα:

- **Γρήγορες απαντήσεις.** Ο χρήστης παίρνει άμεσα απάντηση· η αργή δουλειά γίνεται μετά.
- **Ανεξάρτητη κλιμάκωση.** Αν στοιβάζονται εργασίες, προσθέτεις κι άλλους workers χωρίς να αγγίξεις το API.

Συνηθισμένα εργαλεία: Celery ή RQ σε Python, BullMQ σε Node, συνήθως με Redis ή RabbitMQ από κάτω ως ουρά. Τίποτα δεν είναι δωρεάν, όμως: με ουρά, η δουλειά δεν είναι πια άμεση, οπότε χρειάζεσαι τρόπο να ελέγχει ο χρήστης αργότερα την πρόοδο — συνήθως ένα αναγνωριστικό εργασίας που ρωτάει «τελείωσε;».

♦ Έργο — Βήμα 6: μετέφερε το email στο παρασκήνιο

Πάρε το email καλωσορίσματος από το προηγούμενο βήμα και βάλ' το σε μια ουρά. Όταν εγγράφεται ένας χρήστης, στοίβαξε μια εργασία και γύρνα αμέσως απάντηση· ένας worker στέλνει το email στο παρασκήνιο.

Το endpoint εγγραφής σου είναι πλέον γρήγορο, ανεξάρτητα από το πόσο αργός είναι ο πάροχος email εκείνη τη μέρα.

10. Προσωρινή αποθήκευση (caching)

Το caching είναι από τους απλούστερους και πιο αποτελεσματικούς τρόπους να επιταχύνεις ένα backend και να ελαφρύνεις τη βάση σου. Η ιδέα: αποθήκευσε τα δεδομένα που ζητιούνται συχνά κάπου εξαιρετικά γρήγορα (συνήθως **Redis**, μια βάση που ζει στη μνήμη), ώστε να μη χρειάζεται να κάνεις την αργή ανάκτηση κάθε φορά.

Το πρόβλημα — και πάντα υπάρχει ένα — είναι η **ακύρωση του cache**: το να ξέρεις πότε τα αποθηκευμένα δεδομένα έχουν παλιώσει και πρέπει να ανανεωθούν. Αν κρατάς στο cache τη λίστα εξόδων ενός χρήστη και εκείνος προσθέσει ένα νέο, το cache είναι πλέον λάθος μέχρι να το διορθώσεις. Το απλούστερο σημείο εκκίνησης είναι ένα **TTL** (time to live): τα δεδομένα λήγουν αυτόματα μετά από ένα ορισμένο διάστημα — π.χ. 60 δευτερόλεπτα. Εύκολο και αρκετά καλό για τεράστιο εύρος περιπτώσεων.

Ο εμπειρικός κανόνας: αποθήκευε προσωρινά μόνο δεδομένα που διαβάζονται συχνά και αλλάζουν σπάνια. Το να κάνεις cache δεδομένα που αλλάζουν κάθε δευτερόλεπτο απλώς σερβίρει παλιά στοιχεία και προσθέτει πολυπλοκότητα χωρίς όφελος.

◆ Έργο — Βήμα 7: βάλε cache στη λίστα εξόδων

Αποθήκευσε προσωρινά τη λίστα εξόδων ενός χρήστη στο Redis με σύντομο TTL. Σε κάθε ανάγνωση, έλεγξε πρώτα το cache· αν λείπει, διάβασε από την Postgres και αποθήκευσε το αποτέλεσμα.

Ακύρωσε ή ανανέωσε το cache όταν ο χρήστης προσθέτει ή αλλάζει ένα έξοδο, ώστε να μη βλέπει ποτέ παλιά δεδομένα. Μετά δες τις επαναλαμβανόμενες αναγνώσεις να γίνονται αισθητά γρηγορότερες.

11. Δοκιμές (testing)

Ο κώδικας χωρίς δοκιμές είναι κώδικας που απλώς δεν έχει σπάσει ακόμη. Οι **δοκιμές** είναι μικρά προγράμματα που ελέγχουν αυτόματα ότι ο κώδικας σου κάνει αυτό που νομίζεις. Είναι αυτό που σου επιτρέπει να αλλάζεις το backend σου χωρίς να ξενυχτάς αναρωτώμενος τι χάλασες σιωπηλά.

Υπάρχουν μερικά είδη, και αξίζει να καταλάβεις τη διαφορά:

Είδος	Τι δοκιμάζει	Ταχύτητα
Μονάδας (unit)	Μία μικρή λειτουργία απομονωμένα	Πολύ γρήγορο
Ολοκλήρωσης (integration)	Πολλά κομμάτια μαζί (API + βάση)	Πιο αργό
Άκρο-σε-άκρο (end-to-end)	Όλη η ροή όπως θα την ζούσε ένας χρήστης	Πιο αργό

Η πρακτική εικόνα είναι μια «πυραμίδα»: πολλές γρήγορες δοκιμές μονάδας στη βάση, μερικές δοκιμές ολοκλήρωσης στη μέση, ελάχιστες αργές end-to-end στην κορυφή. Δεν χρειάζεται να δοκιμάζεις κάθε γραμμή — το κυνήγι για 100% κάλυψη είναι χάσιμο χρόνου. Κάλυψε τις **κρίσιμες διαδρομές**: εγγραφή, σύνδεση, δημιουργία εξόδου και έλεγχο πρόσβασης.

Χρησιμοποίησε τα τυπικά εργαλεία της γλώσσας σου: pytest για Python, JUnit για Java, Jest για Node.

♦ Έργο — Βήμα 8: δοκίμασε τον κώδικά σου

Γράψε δοκιμές μονάδας για τη βασική λογική και δοκιμές ολοκλήρωσης για τα κύρια endpoints. Τουλάχιστον απόδειξε ότι η εγγραφή και η σύνδεση δουλεύουν, ότι ένας συνδεδεμένος χρήστης μπορεί να δημιουργήσει ένα έξοδο, και — κρίσιμο — ότι ένας χρήστης δεν μπορεί με κανέναν τρόπο να διαβάσει ή να αλλάξει τα έξοδα κάποιου άλλου.

Οδηγός: ενότητα testing στην επίσημη τεκμηρίωση FastAPI.

12. Δημοσίευση και παρατηρησιμότητα

Ένα backend που τρέχει μόνο στον φορητό σου είναι χόμπι. Η **δημοσίευση (deployment)** — το να το βάλεις σε έναν server που είναι πάντα ανοιχτός ώστε να το χρησιμοποιεί ο καθένας — το κάνει αληθινό. Η **παρατηρησιμότητα** είναι το πώς ξέρεις ότι είναι ακόμη ζωντανό και υγιές εκεί έξω, όπου δεν μπορείς να το κοιτάς απευθείας.

Δημοσίευση

Το μεγάλο πρόβλημα που λύνει η δημοσίευση: κώδικας που δουλεύει στο μηχανήμα σου συχνά σκάει σε άλλο, επειδή κάτι είναι ρυθμισμένο διαφορετικά. Η λύση είναι το **Docker**, που συσκευάζει την εφαρμογή σου και όλα όσα χρειάζεται σε ένα **container** που τρέχει

παντού ολόιδια. Το φτιάχνεις μία φορά και συμπεριφέρεται το ίδιο στον φορητό σου και στο cloud.

Δεν χρειάζεσαι Kubernetes ούτε τίποτα τρομακτικό την πρώτη μέρα. Ξεκίνα απλά:

- Μια φιλική πλατφόρμα όπως Railway, Render ή Fly.io, που παίρνει το container σου και το τρέχει online για σένα.
- Μια **διαχειριζόμενη Postgres** (η πλατφόρμα τρέχει τη βάση για σένα) αντί να τη συντηρείς μόνος σου.
- Μεταβλητές περιβάλλοντος για όλες τις ρυθμίσεις και τα μυστικά, ορισμένες στον πίνακα ελέγχου της πλατφόρμας.



Παρατηρησιμότητα

Μόλις βγει στον αέρα, δεν μπορείς απλώς να κοιτάς το τερματικό σου. Η παρατηρησιμότητα σου δίνει μάτια. Τρεις πυλώνες:

- **Logs (αρχεία καταγραφής):** γραπτό ιστορικό του τι συνέβη (το έχεις ήδη στήσει).
- **Metrics (μετρήσεις):** αριθμοί στον χρόνο — αιτήματα ανά δευτερόλεπτο, ποσοστό σφαλμάτων, χρόνος απόκρισης.
- **Alerts (ειδοποιήσεις):** αυτόματες προειδοποιήσεις όταν κάτι πάει στραβά, πριν το προσέξουν οι χρήστες σου.

Δομημένα logs συν μία βασική ειδοποίηση (π.χ. «πες μου αν το ποσοστό σφαλμάτων 5xx εκτιναχθεί ξαφνικά») σε βάζουν ήδη μπροστά από έναν εκπληκτικό αριθμό πραγματικών υπηρεσιών παραγωγής.

♦ Έργο — Βήμα 9: βγάλ' το στον αέρα

Κάνε τον καταγραφέα εξόδων container με Docker. Δημοσίευσέ τον σε Railway, Render ή Fly.io με διαχειριζόμενη Postgres και Redis. Επιβεβαίωσε ότι μπορείς να συνδεθείς στο ζωντανό API σου από το κινητό σου, από άλλο δίκτυο, από οπουδήποτε.

Πρόσθεσε δομημένη καταγραφή και μια βασική ειδοποίηση για όταν ανεβαίνει το ποσοστό σφαλμάτων. Στείλ' το σε έναν φίλο και ζήτα του να το δοκιμάσει.

Τι κάνεις μετά

Αν έφτασες ως εδώ χτίζοντας τον καταγραφέα εξόδων βήμα προς βήμα — αντί απλώς να διαβάζεις — τότε δεν κάνεις πια «μαθήματα» για το backend· έχεις φτιάξει ένα. Έχεις ένα ολοκληρωμένο, δημοσιευμένο, δοκιμασμένο backend με ταυτοποίηση, πραγματική βάση δεδομένων, αποθήκευση αρχείων, εργασίες παρασκηνίου, cache και παρακολούθηση. Αυτό είναι το σχήμα μιας αληθινής υπηρεσίας παραγωγής, και το έφτιαξες μόνος σου.

Το επόμενο βήμα είναι βάθος και επανάληψη. Δύο κατευθύνσεις αξίζουν:

- **System design.** Αυτός ο οδηγός σου έδειξε να χτίζεις μια υπηρεσία· ο σχεδιασμός συστημάτων σου δείχνει πώς κρατάς πράγματα όρθια σε κλίμακα — πότε να διαλέξεις ισχυρή έναντι τελικής συνέπειας, πώς σχεδιάζεις για εκατομμύρια χρήστες αντί για έναν.
- **Επανάληψη με νέα έργα.** Ξαναφτιάξε τα ίδια θεμέλια σε ένα διαφορετικό πρόβλημα — ένα API για σημειώσεις, έναν συντομευτή συνδέσμων, ένα μικρό ηλεκτρονικό κατάστημα. Την τρίτη φορά, τα μοτίβα θα σου φαίνονται φυσικά.

Και πάνω απ' όλα: συνέχισε να χτίζεις κάτι που τρέχει αληθινά. Η θεωρία χωρίς αποστολή ξεθωριάζει· ένα ζωντανό σύστημα που μπορείς να δείξεις μένει.

Οδηγός του **Ιωάννη Βόγγελη**, που διατίθεται δωρεάν στο πλαίσιο του προσωπικού του brand για εκπαιδευτική χρήση. Το εκπαιδευτικό περιεχόμενο και το παράδειγμα του έργου (καταγραφέας εξόδων) είναι πρωτότυπα· τα τεχνικά εργαλεία και οι υπηρεσίες που αναφέρονται ανήκουν στους αντίστοιχους κατόχους τους.